DIGITAL STUDENT INFORMATION SYSTEM

Introduction to Computing Project

Presented to the
Faculty of the College of Information Technology
Cagayan de Oro College
Cagayan de Oro City

In Partial Fulfillment
of the Requirements for the Degree

Bachelor of Science in Information Technology

Proponents

KILATON, RICH GLENN R.
GARCIA, NATHAN JHON B.
AMIHAN, JOHN PAUL B.
QUIPIT, ARMAND CARLO S.
MAHUSAY, JELORD U.
AGUILAR, JUDY ANN N.
FELISARTA, JECKY R.

October 2026

1.1 Introduction

A student information system (SIS) is an application that allows a school to manage student data, such as personal details, attendance, and class schedules, in a single place. Mo (2024) explained that handling student information through an automated system reduces manual operations and errors and makes services such as information inquiry and course scheduling easier to offer to students. Similarly, Resare and Bautista (2014) described a student records information system as an application that enables a school to enter student details, build student records, and manage other student-related data, while making information faster to pass along and access. These observations are relevant to this project because registrars, instructors, and students all depend on the same records, and keeping those records organized and easy to retrieve is a basic information-management need of any school.

For this project, the proponents assume a school in which the registrar's office and instructors keep student records, attendance, and class schedules separately, such as on paper or in separate files, and in which students must ask staff to learn about their own records. No actual client was interviewed, and no data were collected from a real school. The sources cited here describe other organizations; therefore, they support only possible problems and do not prove that these problems occur in the assumed school. Muthanna Kanona (2022), for example, reported that the student affairs of one college were handled mostly through paper-based activities, which suggests that a school that keeps records manually may face similar difficulties. Based on Mo (2024), Resare and Bautista (2014), and Muthanna Kanona (2022), the project considers three possible difficulties: inconsistent or duplicate student records, difficulty in accessing attendance records, and outdated or unclear class schedules.

The proponents developed the Digital Student Information System as a Python console-based application to help registrars, instructors, and students manage student information. The system allows registrars to add students (ID, name, course or field, birthday, and status); to view, search, update, and remove student records; and to manage class schedules. It allows instructors to add or update attendance and class schedules, view full student records, and search for students. It allows students to log in and view their personal information, attendance, and class schedule. Records are displayed in formatted console tables, and each portal shows the current date and time.

The ID, name, and birthday checks and the duplicate-ID check address the first problem; the validated attendance entries, the computed attendance rate, and the student portal address the second; and the schedule functions with their format checks address the third, within the limits described in the Scope and Limitations.

1.2 Statement of the Problem

General Problem

In the assumed school, student records, attendance, and class schedules are handled in separate places and without consistent checking. Registrars, instructors, and students are all affected, and the practical result is records that are difficult to verify, retrieve, and keep current.

Specific Problems

The study specifically addresses the following problems:

1. Student records may contain duplicate IDs, invalid names, or incomplete details such as the course and birthday, which makes it difficult to identify the correct student.
2. Attendance records are difficult for students to view and for instructors to record consistently, which can lead to incorrect or unavailable information.
3. Class schedules are difficult to update and view, which can leave students with outdated or unclear schedules.

1.3 Objectives of the Study

General Objective

To develop the Digital Student Information System, a Python console-based application that enables registrars, instructors, and students to manage and view student records, attendance, and class schedules.

Specific Objectives

The study specifically aims to:

1. Record, update, search, and remove student records (ID, name, course or field, birthday, and status), with validation that rejects duplicate or incorrectly formatted student IDs, names not written as Last Name, First Name Middle Initial, and invalid birthdays;
2. Record and display attendance per subject, with validation that entries are whole numbers and that the present and absent days equal the total days, and with the attendance rate computed as a percentage; and
3. Add, update, delete, and display class schedules for each student, with validation of the days and of the start and end times.

1.4 Scope

The Digital Student Information System is a Python console-based application for an assumed school setting with three user roles: registrar, instructor, and student. It runs as a text-based numbered menu on one local computer, is operated by one user at a time, displays records in formatted console tables, and stores all records in Python lists held in memory while the program is running. The system covers the following:

- **Registrar:** adds students (ID, name, course or field, birthday, and status); views a student's full record; searches students; updates a student's ID, name, course or field, birthday, or status; adds, updates, deletes, or views a student's class schedule; and removes a student after confirmation.
- **Instructor:** adds or updates a student's attendance per subject; adds, updates, deletes, or views a student's class schedule; views a student's full record (information, schedule, and attendance); and searches students.
- **Student:** logs in and views personal information (ID, name, course or field, birthday, age, status, and date registered), attendance (total, present, absent, and rate), and class schedule.
- **Any user:** creates a student, registrar, or instructor account with a username of 4 to 30 characters and a password of at least 6 characters, neither containing spaces.
- **Search:** registrars and instructors can find a student by typing part of an ID, name, course or field, or birthday, or by selecting a student from the displayed list of registered students.

- **Data handled:** username, password, student ID, student name, course or field, birthday, age (computed from the birthday), status (Regular or Irregular), date registered, subject, total days, days present, days absent, and schedule text.

1.5 Limitations

All records, including accounts, are stored only in memory; therefore, they are lost when the program closes. The system has no graphical interface, no online access, and no automated reminders or notifications, and it does not support simultaneous use across devices because users take turns on one computer. Login uses basic username and password comparison only; passwords are stored as plain text, and no encryption or advanced authentication is implemented. In addition, the system has the following limitations:

- Anyone using the program can create a registrar or instructor account, as there is no approval process.
- A student account is not linked to a student ID. After logging in, the student types a Student ID, and the program does not verify that the ID belongs to that account.
- Attendance is stored as totals per subject (total, present, and absent days) rather than as dated daily entries, and the program does not check for schedule conflicts between subjects.
- Only students can be removed, and only individual class schedules can be deleted, both after confirmation. Deleting attendance records or accounts and changing passwords are not included.

1.6 System Requirements

Hardware Requirements

- **Computer:** A computer capable of running Python 3. The program keeps its records in memory and uses only a text console; no minimum processor, memory, or storage specifications were determined.
- **Display and input:** A display and a keyboard.
- **Network:** None. The program does not use network access.

Software Requirements

- **Operating system:** Windows 10
- **Programming language:** Python
- **Development environment:** Visual Studio Code

Functional Requirements

Registrar

- The registrar can add a student by entering a student ID, name, course or field, birthday, and status. The program rejects an empty or incorrectly formatted ID (accepted: 1 to 8 digits or the format 2024-0001), a duplicate ID, a name not written as Last Name, First Name Middle Initial, an invalid course or field, a birthday that is not in MM/DD/YYYY format, is in the future, or gives an age outside 5 to 100, and an invalid status. The program standardizes the capitalization of the name and records the date registered.
- The registrar can update a student's ID, name, course or field, birthday, or status. The program applies the same checks as when adding a student and rejects a new ID that already exists.
- The registrar can add, update, delete, and view a student's class schedule. The program deletes a schedule only after confirmation and rejects a schedule that is not in the format Mon, Wed, Fri 08:00 AM - 09:30 AM, an invalid day or time, an end time that is not later than the start time, and a subject that is already scheduled for the student.
- The registrar can view a student's full record by selecting the student from the displayed list or by searching for the student.
- The registrar can remove a student only after answering Y (yes) to the confirmation prompt. Removing a student also deletes that student's schedule and attendance records.

Instructor

- The instructor can add or update attendance for a subject. The program accepts whole numbers only, requires at least one total day, and rejects totals in which present plus absent days do not equal total days. Subject names are standardized and matched regardless of capitalization, and the updated attendance table is displayed after saving.

- The instructor can add, update, delete, and view a student's class schedule, with the same schedule format checks and deletion confirmation used by the registrar.
- The instructor can view a student's full record (information, schedule, and attendance) and search for students.

Student

- The student can log in, enter their Student ID, and view personal information (including course or field, birthday, age, status, and date registered), attendance with the rate, and class schedule.

General Rules

- The program rejects usernames that are not 4 to 30 characters long or that contain spaces, passwords shorter than 6 characters or containing spaces, duplicate usernames within a role (regardless of capitalization), and invalid menu choices.

1.7 Code Snippets of the Digital Student Information System

The following snippets show the main functions of the system and the objectives that each one supports.

Snippet 1: Add Student with Validation (add_student)

Related objective: *Specific Objective 1: Record, update, search, and remove student records.*

```
def add_student():
    title("ADD STUDENT")
    while True:
        student_id = ask_student_id()
        if student_id is None:
            return
        if find_student(student_id) == -1:
            break
        print("ID already exists. Enter a different ID.")
    name = ask_name()
    if name is None:
        print("Cancelled.")
        pause()
        return
    course = ask_course()
    if course is None:
        print("Cancelled.")
        pause()
        return
    print()
    birthday = ask_birthday()
    if birthday is None:
        print("Cancelled.")
        pause()
        return
    print("\n1. Regular")
    print("2. Irregular")
    choice = ask_choice("Choose status (ENTER = cancel): ", ("1", "2"), True)
    if choice is None:
        print("Cancelled.")
        pause()
        return
    status = "Regular" if choice == "1" else "Irregular"
    student_ids.append(student_id)
    student_names.append(name)
    student_courses.append(course)
    student_birthdays.append(birthday)
    student_status.append(status)
    student_registered.append(datetime.now().strftime("%B %d, %Y"))
    student_schedules.append([])
    student_attendance.append([])
    print("\nStudent added!")
    display_student(len(student_ids) - 1)
    pause()
```

The function prompts the registrar for a student ID, name, course or field, birthday, and status, one at a time. It repeats each prompt until the entry is valid; the ID must also be unused. The function stops if the registrar cancels. Once all entries are accepted, it appends the record, together with the date registered and empty schedule and attendance lists, and displays the new student record. This supports Objective 1.

Snippet 2: Record Attendance with Validation (manage_attendance)

Related objective: *Specific Objective 2: Record and display attendance per subject, with validation.*

```
def manage_attendance():
    title("ATTENDANCE MANAGEMENT")
    index = pick_student()
    if index == -1:
        return
    print("\nStudent:", student_names[index])
    subject = ask_subject()
    if subject is None:
        print("Cancelled.")
        pause()
        return
    print("\nEnter Attendance")
    numbers = ask_attendance_numbers()
    if numbers is None:
        print("Cancelled.")
        pause()
        return
    total_days, present, absent = numbers
    found = False
    for record in student_attendance[index]:
        if record[0].lower() == subject.lower():
            record[1] = total_days
            record[2] = present
            record[3] = absent
            found = True
    if found:
        print("\nAttendance updated.")
    else:
        student_attendance[index].append(
            [subject, total_days, present, absent]
        )
        print("\nAttendance added.")
    print("\nCurrent attendance for", student_names[index])
    show_attendance(index)
    pause()
```

The function lets the instructor select a student and enter a subject, and then collects the total, present, and absent days through the `ask_attendance_numbers` helper. That helper accepts whole numbers only, requires at least one total day, and prompts again if present plus absent days

do not equal the total days. The function then updates the subject's existing record or adds a new one and displays the attendance table with each subject's rate. This supports Objective 2.

Snippet 3: Add and Update Class Schedule (add_schedule and update_schedule)

Related objective: *Specific Objective 3: Add, update, delete, and display class schedules for each student.*

```
def add_schedule(index):
    while True:
        subject = ask_subject()
        if subject is None:
            print("Cancelled.")
            return
        if find_schedule(index, subject) != -1:
            print("Subject already scheduled. Enter a different subject.")
            continue
        schedule = ask_schedule()
        if schedule is None:
            print("Cancelled.")
            return
        student_schedules[index].append(subject + " - " + schedule)
        print("Schedule added.")
        return

def update_schedule(index):
    position = ask_schedule_position(index)
    if position is None:
        return
    new_schedule = ask_schedule("New Schedule")
    if new_schedule is None:
        print("Cancelled.")
        return
    subject = student_schedules[index][position].split(" - ", 1)[0]
    student_schedules[index][position] = subject + " - " + new_schedule
    print("Schedule updated.")
```

The `add_schedule` function asks for a subject and a schedule, refuses a subject that is already scheduled, and appends the entry. The `update_schedule` function lets the user choose an existing entry by number and replaces its schedule text. Both rely on `ask_schedule`, which keeps asking until the text matches a valid format, such as Mon, Wed, Fri 08:00 AM - 09:30 AM, with an end time later than the start time. The saved entries are shown in the student's schedule table. This supports Objective 3.

Snippet 4: Search Students by Keyword (search_students)

Related objective: *Specific Objective 1: Record, update, search, and remove student records.*

```
def search_students(keyword):
    # Every word typed must appear somewhere in the student's
```

```
    # ID, name, course/field, or birthday (case-insensitive).
    words = keyword.lower().split()
    matches = []
    for i in range(len(student_ids)):
        text = " ".join([
            student_ids[i],
            student_names[i],
            student_courses[i],
            course_label(student_courses[i]),
            format_birthday(student_birthdays[i])
        ]).lower()
        if all(word in text for word in words):
            matches.append(i)
    return matches
```

The function splits the typed keyword into words and, for each student, combines the ID, name, course or field code, full course name, and birthday into one lowercase text. A student is a match only when every typed word appears in that text, and the function returns the positions of the matching students. The registrar and instructor menus display these matches in a results table and let the user select one student, so a record can be found without remembering the exact Student ID. This supports Objective 1.

REFERENCES

Mo, B. (2024). Design and implementation of an information management system for college students in higher education institutions based on cloud computing. *International Journal of Advanced Computer Science and Applications, 15*(5), 289–302. https://thesai.org/Downloads/Volume15No5/Paper_30-Design_and_Implementation_of_an_Information_Management_System.pdf

Muthanna Kanona, R. (2022). Student information management system for Baghdad College of Economic Sciences University (SIMS-BC) as: A case study. *Baghdad College of Economic Sciences University Journal (BCESUJ), 67*(1), 316–325. https://ojs.baghdadcollege.edu.iq/index.php/BCESUJ/article/view/120

Resare, P. M. M., & Bautista, M. A. P. (2014). Students' record information system (SRIS) AIMS-CeSS department. *BENCHMARK: College of Business Research Journal, 2*(1). https://ejournals.ph/article.php?id=10424